

LESSON 2: DATA STRUCTURES

In our current digital age, data is everywhere. From a simple image file on a smartphone to a complex database on a Google server, data surrounds us and is essential for the functioning of our apps and the generation of the information we rely on daily. However, data does not exist randomly in our computers; all stored data is organized in specific structures that allow us to organize, retrieve, modify, process, and store it efficiently. In this lesson, we will explore these data structures to gain a better understanding of their roles and importance.

DATA STRUCTURES ARE SPECIALIZED FORMATS FOR ORGANIZING, RETRIEVING, MODIFYING, PROCESSING, AND STORING DATA.

CLASSIFYING DATA STRUCTURES

Data structures are broadly classified based on the arrangement of data items within the structure. When data items are arranged sequentially (e.g., 1st, 2nd, 3rd, and so on) like books on a shelf, such structures are referred to as linear, such as lists or arrays.

EXAMPLE 2.1

Data in a linear structure are arranged sequentially.



"13, 47, 25, 28, 5, 32"

"Dana, Alex, Maya, Omar, Lana, Chen"

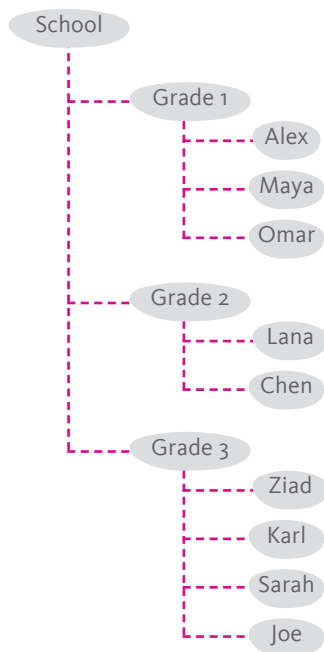
13	47	25	28	5	32
1 st	2 nd	3 rd	4 th	5 th	6 th

Dana	Alex	Maya	Omar	Lana	Chen
1 st	2 nd	3 rd	4 th	5 th	6 th

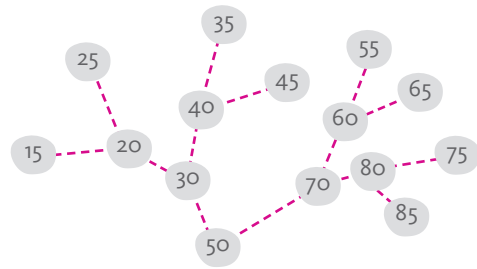
DATA STRUCTURES ARE CLASSIFIED AS LINEAR AND NON-LINEAR.

However, data items can be arranged in other ways. One such way is as a hierarchy, forming a structure similar to a tree, and therefore referred to as a tree data structure. In a tree, data is arranged along branches. Each branching point is referred to as a node, starting from the 'root node'. Each branching results in a new level in the hierarchy, with the root node existing at the first level.

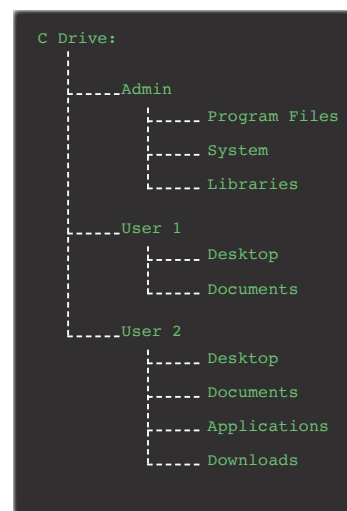
EXAMPLE 2.2



This organization chart for a school is a tree data structure with three hierarchical levels.



A binary search tree is used for efficiently searching databases and directories. This one has four levels. Can you count them?



This computer directory is a tree data structure with three hierarchical levels.

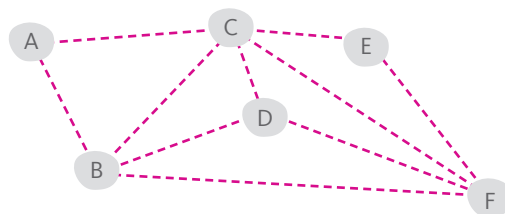
In the tree data structure, data items are connected along branches, and items in one branch cannot connect to items in another branch. However, sometimes it is necessary for data items to freely interconnect so that any data item can connect to any other data item, forming a structure known as a graph data structure.

In a graph, each data point acts as a node and can point to one or more other nodes in the structure. Information about the link between two nodes, referred to as an edge, is also stored as data information at each node. This information about the edge could represent the distance between the nodes, the time it takes to travel between the two nodes, or other relevant metrics.

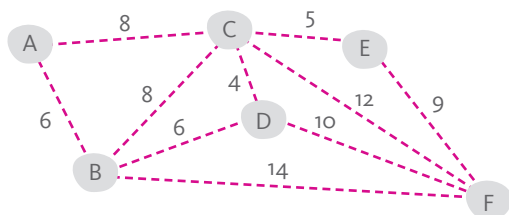
EXAMPLE 2.3



A map of flight routes between cities.



A graph data structure used to digitally represent the flight routes, with each city defined as a node.



The graph identifies the weight of each edge, with the number representing the hours of flight between two cities.

Adjacency List:

A: [(B, 6), (C, 8)]
 B: [(A, 6), (C, 8), (D, 6), (F, 14)]
 C: [(A, 8), (B, 8), (D, 4), (E, 5), (F, 12)]
 D: [(B, 6), (C, 4), (F, 10)]
 E: [(C, 5), (F, 9)]
 F: [(B, 14), (C, 12), (D, 10), (E, 9)]

This is an adjacency list to represent the graph. For each node, the nodes connected to it are listed along with their edge weights.

ALGORITHMS & DATA STRUCTURES

Everything inside a computer is determined by instructions, or algorithms—sets of instructions specific for particular tasks. One such group of tasks involves controlling data structures. Algorithms control data structures. These algorithms define the rules for size, data type, and methods for adding, removing, and accessing items in various data structures, whether linear (like lists), hierarchical (like trees), or interconnected (like graphs).

ALGORITHMS CONTROL DATA STRUCTURES.

LINEAR STRUCTURES

There are five common linear data structures: the array, list, tuple, queue, and stack. Being linear structures, all five maintain their items in a sequential manner. However, they vary depending on the rules that determine their size, the type of data they can accept, and how items are inserted or removed.

THE ARRAY

The array is unique because it is defined by a fixed size and a homogeneous set of data. Once an array is created, the number of positions it contains remains constant. While the data items within an array can be changed or swapped, no new positions can be added or removed, maintaining the array's fixed size. Additionally, an array can only hold one type of data, meaning all items must be of the same type, either numeric or alphanumeric.

	Array	List	Stack	Queue
Size	Fixed	Dynamic	Dynamic	Dynamic
Data type accepted	one type	mixed	mixed	mixed
Insert data	not allowed	any position	end	end
Remove data	not allowed	any position	end	beginning

In contrast, the list, stack, and queue are dynamic data structures. This means that their sizes can change as data is inserted or removed. They are also more flexible regarding the types of data they can contain, allowing for a mixture of data types.

EXAMPLE 2.4

Maya, Sarah, and Leen are three sisters. Maya is 12 years old, Sarah is 9, and Leen is 14. If we wanted to use an array as a data structure to store their names and ages, we would need to use two separate arrays: one for the alphanumeric (string) data, which would store their names, and another for the numeric (number) data, which would store their ages. This is because an array cannot hold more than one type of data simultaneously.



The three sisters: Maya, Sarah, and Leen.

```
name_array = ["Maya", "Sarah", "Leen"]  
age_array = [12, 7, 14]
```

Instructions for creating two arrays one for each type of data.

```
CREATE New Dataset  
  Structure = Linear  
  Size = Fixed  
  Data Type = Numeric  
  Elements = [12, 7, 14]  
  Name = "age_array"  
  Insert = Not Allowed  
  Remove = Not Allowed
```

A hypothetical algorithm in pseudocode that would operate behind the scenes to create the 'age_array'.

Let's assume there was an error with Sarah's age—she is actually 9 years old, not 7. Although we cannot insert or remove data in an array, as that would change the array's size, we can swap out or modify the existing data items. In this case, we would simply update the value at the position corresponding to Sarah's age, correcting it from 7 to 9.

```
age_array = [12, 7, 14]
    UPDATE index (1) = 9
    PRINT age_array
END
```

OUTPUT:
12, 9, 14

The pseudocode instructions and the output for changing the value of the data held at position (1) in the array.

Note: Positions in a linear data structure are indexed, meaning each position is assigned a sequential number. Indexing typically starts at 0, so the first position in the structure has an index of 0.

THE LIST

A list is another linear data structure that, unlike an array, has a dynamic size and can accept a mixture of data types—numeric, alphanumeric, or both. Being dynamic in size means that it can change by inserting or removing data items. In a list, this insertion and removal of data items can occur at any position.

EXAMPLE 2.5

Jamal is Maya, Sarah, and Leen's 11-year-old brother. To add his name to the 'name_array' or his age to the 'age_array' is not possible, as that would change the size of the arrays. However, if these arrays were lists, then adding Jamal and his age would be possible.



Jamal is 11 years old.

```

name_list = ["Maya", "Sarah", "Leen"]
    INSERT index (2) = "Jamal"
    PRINT name_list
END

```

OUTPUT:
Maya, Sarah, Jamal, Leen

```

age_list = [12, 9, 14]
    INSERT index (2) = 11
    PRINT age_list
END

```

OUTPUT:
12, 9, 11, 14

Two sets of pseudocode instructions, each creating a list, inserting a new item at the third position (2), and then printing the list. Inserting a new item shifts the current items up by one position.

Note that each list contains only one data type, even though a list can simultaneously accept multiple data types.

If we want to sort the children in ascending order from youngest to oldest, we would need to shift the data items “Maya” and her age, 12, from their current positions (0) to the third position (2), currently occupied by “Jamal” and his age, 11.

```

age_list = [12, 9, 11, 14]
    REMOVE index (0) = 12
    INSERT index (2) = 12
    PRINT age_list
END

```

OUTPUT:
9, 11, 12, 14

```

name_list = ["Maya", "Sarah", "Jamal", "Leen"]
    REMOVE index (0) = "Maya"
    INSERT index (2) = "Maya"
    PRINT name_list
END

```

OUTPUT:
Sarah, Jamal, Maya, Leen

The two sets of pseudocode instructions show that we first sort the ‘age_list’ by changing the position of data item 12 from index (0) to (2). Then, the second set of instructions does the same with the name “Maya.” Now, both lists are sorted.

The last example, while it demonstrates how easily a list lends itself to the insertion and removal of data items, also reveals a limitation: when dealing with two related datasets, a list does not allow for associating between related data items, such as the name and age of each sibling. Even if we add both names and ages to one list, the related data items cannot be inherently associated and manipulated together.

THE TUPLE

The tuple is yet another linear data structure. It is similar to an array in that it has a fixed size, but it can accept different data types. The more interesting feature of a tuple is that it allows each position to be associated with specific data, effectively allowing related data items to be associated together.

EXAMPLE 2.6

Tuples can be used to combine the name and age of each sibling into one data unit with the format (name, age). Using such tuples would make the list more organized and make data manipulation easier. We have used brackets '[']' to define arrays and lists, whereas a tuple will be defined using parentheses '(')'.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]  
    PRINT sibling_list  
END
```

OUTPUT:

Maya, 12, Sarah, 9, Jamal, 11, Leen, 14

Using tuples to organize related datasets in a list ensures that the related data items, such as the name and age of each sibling, follow each other. This method helps maintain the association between data points and makes it easier to manage and manipulate the data within the list.

Now, if we want to sort the children in ascending order from youngest to oldest, we would still need to shift the data items “Maya” and her age, 12, from their current positions (0) to the third position (2), currently occupied by “Jamal” and his age, 11. However, since the data is now organized by tuples, we simply need to move the entire tuple containing Maya’s data from its current position (0) to occupy the third position (2), which currently holds the tuple with Jamal’s data.

This approach makes the sorting process more straightforward, as related data points remain together within each tuple, reducing the complexity of shifting multiple individual data items.

```
sibling_list = [("Maya", 12), ("Sarah", 9), ("Jamal", 11), ("Leen", 14)]
REMOVE index (0) = ("Maya", 12)
INSERT index (2) = ("Maya", 12)
PRINT sibling_list
END
```

OUTPUT:

Sarah, 9, Jamal, 11, Maya, 12, Leen, 14

These instructions create a list of tuples, each formatted as (name, age). The data items in each tuple are fixed together in the order in which they were placed, with the name at index (0) and the age at index (1) of the tuple. The tuples are then placed in a list, with each tuple occupying one position in the list as if it were a single data item. So when Maya’s data needs to be moved, it is moved together as a tuple from its position at index (0) to its new position at index (2).

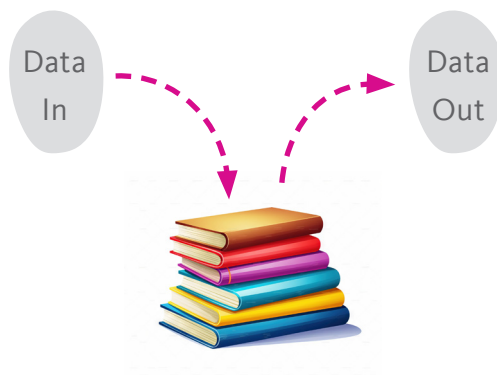
Notes

Creating a list of tuples in the same format is akin to creating a table, with each tuple acting as an entry on a row and the data items in each tuple placed according to their corresponding field or column.

THE STACK

The stack is similar to a list in that it has a dynamic size, meaning we can add to and remove items from it as needed, and it can accept data of different types, including both strings and numbers. The main difference is that in a stack, inserting and removing data items can only happen from the end of the stack. As a result, the last item added to the stack will be the first item to be removed; this is referred to as the Last In, First Out (LIFO) principle.

THE LIFO PRINCIPLE STATES THAT IN AN ORDERED SET OF ITEMS, THE LAST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.



In a stack of books, the first book that can be removed is the last book that was added to the stack.

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = ["A", "B", "C"]
  Name = "file_stack"
  Insert = To End
  Remove = From End
```

A hypothetical algorithm in pseudo-code that would operate behind the scenes to create a stack.

EXAMPLE 2.7

Lee works at a warehouse for an electronics store, where he maintains an up-to-date inventory of all products in stock. As products frequently move in and out of the warehouse, Lee must keep an accurate list of the current inventory.



One of the products in the warehouse is television sets. Each inbound TV is assigned a unique SKU code, which is entered into the warehouse computer system before the TV is placed on the shelf with other TVs waiting to be sold. The TVs are typically stacked, with the most recent arrival placed on top. When a client buys a TV, the one at the top of the stack is selected and shipped out to the client.

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = []
  Name = "tv_stack"
  Insert = To End
  Remove = From End
```

OUTPUT:
tv_stack = []

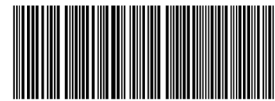
The hypothetical pseudocode instructions would create a stack, a linear data structure that can grow to any length and is assigned the name 'tv_stack'.

It is initially empty and does not contain any data elements. Insertion and removal of data is defined to occur only at the end of the stack.

```
IF tv = inbound THEN
  OPEN tv_stack
  READ tv_sku
  INSERT index (END) = tv_sku
  SEND tv TO shelf
END
```

OUTPUT:
tv_stack = [
 "TV-55S4K-BLK-342"
]

When the first TV arrives at the warehouse, the computer system checks to confirm it is inbound. It reads the SKU barcode, 'tv_sku', and adds it to the end of the stack, 'tv_stack', which in this case is the first position since no other TVs have entered the warehouse. The TV is then placed on a shelf.



SKU: TV-55S4K-BLK-342

OUTPUT:
tv_stack = [
 "TV-55S4K-BLK-342"
 "TV-55S4K-BLK-110"
 "TV-55S4K-BLK-729"
 "TV-55S4K-BLK-845"
]

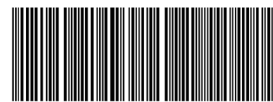
As more TVs arrive at the warehouse, the same process is repeated, and each new 'tv_sku' is added to the end of the stack 'tv_stack'.

Now that a client has purchased a TV, the warehouse needs to ship it out. The computer system receives the order, identifies the last TV set brought in, uses that TV's SKU to locate it on the shelf, and sends it for shipping to the client. This means the last TV set to arrive will be the first to go out, following the LIFO (Last In, First Out) principle.

OUTPUT:

```
tv_stack = [
    "TV-55S4K-BLK-342"
    "TV-55S4K-BLK-110"
    "TV-55S4K-BLK-729"
    "TV-55S4K-BLK-845"
]
```

The last 'tv_sku' is "TV-55S4K-BLK-845".



SKU: TV-55S4K-BLK-845

```
IF tv = outbound THEN
    OPEN tv_stack
    REMOVE index (END) = tv_sku
    FIND tv = tv_sku
    SEND tv TO client
END
```

OUTPUT:

```
tv_stack = [
    "TV-55S4K-BLK-342"
    "TV-55S4K-BLK-110"
    "TV-55S4K-BLK-729"
]
```

The computer system will open the 'tv_stack', check the last entry, remove that SKU, locate the TV with that SKU, and send it for delivery to the client.

Now, if another TV is sold, the next TV to leave the shelf will be the one with the SKU "TV-55S4K-BLK-729."

A data structure should be selected to match the nature of the data and the specific objectives for its use. When a real-world scenario operates under the Last In, First Out (LIFO) principle, a LIFO-based data structure, like a stack, is the most appropriate choice to effectively manage its data. In the electronics store warehouse, where the last product in is typically the first product out, the best data structure to manage the warehouse data is a stack.

THE QUEUE

The queue is similar to the stack and the list in that it has a dynamic size and can accept data of different types, including both strings and numbers. However, the difference lies in the rules that govern the insertion and removal of data. While in a list, these operations can occur at any position, and in a stack, they occur only at the end, the queue follows the First In, First Out (FIFO) principle. This means that the first item added to the queue will be the first item to be removed.

THE **FIFO PRINCIPLE** STATES THAT IN AN ORDERED SET OF ITEMS, THE FIRST ITEM ADDED WILL BE THE FIRST ITEM REMOVED.



In a queue of people at the grocery store, the first person to stand in line is the first person served at the counter. Similarly, in a queue of data, this implies that data can only be removed from the beginning and inserted at the end.

EXAMPLE 2.8

Jenna works at a warehouse for a grocery store. Unlike Lee, who works for an electronics store, many of the products Jenna handles are perishable and have a limited shelf life, such as apples, and should be shipped out as soon as possible before they go bad. Therefore, the first products that come in should be the first to go out.



A crate of apples

```
CREATE New Dataset
  Structure = Linear
  Size = Dynamic
  Data Type = Any
  Elements = [ ]
  Name = "apple_queue"
  Insert = To End
  Remove = From Beginning
```

OUTPUT:
apple_queue = []

```
IF apple_crate = inbound THEN
  OPEN apple_queue
  READ crate_sku
  INSERT index (END) = crate_sku
  SEND apple_crate TO shelf
END
```

OUTPUT:
apple_queue = [
 "APL-1025-20M-012"
]

OUTPUT:
apple_queue = [
 "APL-1025-20M-012"
 "APL-1025-20M-327"
 "APL-1025-20M-445"
 "APL-1025-20M-129"
]

The hypothetical pseudocode instructions would create a queue, a linear data structure that can grow to any length and is assigned the name `apple_queue`.

It is initially empty and does not contain any data elements. Insertion is defined to occur at the end, while removal occurs from the beginning of the queue.

When the first crate of apples arrives at the storage facility, the computer system checks to confirm it is inbound. It reads the SKU barcode on the crate, 'crate_sku', and adds it to the end of the queue, 'apple_queue', which in this case is the first position since this is the first inbound apple crate. The apple crate is then placed in a storage.



SKU: APL-1025-20M-012

As more apple crates arrive, the same process is repeated, and each new 'crate_sku' is added to the end of the queue 'apple_queue'.

Now that a client has purchased two crates of apples, they need to be shipped out. The computer system receives the order, identifies the first two apple crates that arrived, uses those crate SKUs to locate them in the storage area, and sends the crates for shipping to the client. This means the first two apple crates to arrive will be the first ones to go out, following the FIFO (First In, First Out) principle.

OUTPUT:

```
apple_queue = [
    "APL-1025-20M-012"
    "APL-1025-20M-327"
    "APL-1025-20M-445"
    "APL-1025-20M-129"
]
```

The first two 'crate_sku' codes are: "APL-1025-20M-012" and "APL-1025-20M-327".



SKU: APL-1025-20M-012



SKU: "APL-1025-20M-327"

```
IF apple_crate = outbound THEN
    OPEN apple_queue
    REMOVE index (END) = crate_sku
    FIND apple_crate = crate_sku
    SEND apple_crate TO client
END
```

OUTPUT:

```
apple_queue = [
    "APL-1025-20M-445"
    "APL-1025-20M-129"
]
```

The computer system will open the `apple\_queue`, check the first two entries, remove that SKU codes, locate the apple crates with those SKU codes, and send them for delivery to the client.

Now, if another apple crate is sold, the next apple crate to leave the storage will be the one with the SKU "APL-1025-20M-445".

At the electronics store, it was more efficient to remove the TV set on top of the stack to save time, rather than shifting all the TV sets to reach the bottom or the first one that came in. In this scenario, using a FIFO approach would waste time. However, with the apples, it was crucial to get the oldest apples out of storage first, as the longer they remain, the more likely they are to spoil; a LIFO approach would not have worked.

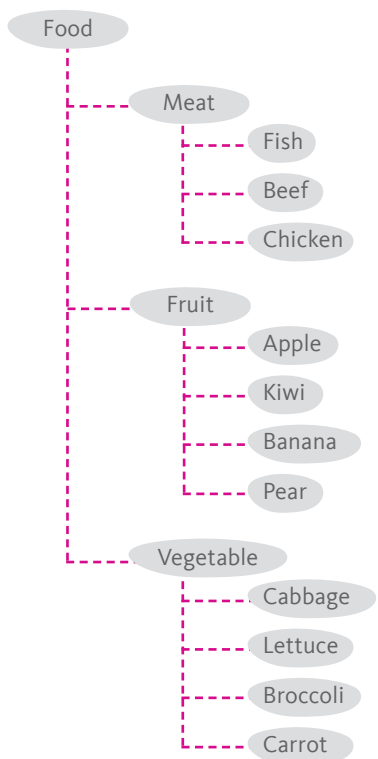
This illustrates that data structures are not just digital constructs but are essential tools designed to meet specific real-world needs. Choosing the right data structure depends on the context and the objectives, whether it's maximizing efficiency or ensuring the freshness of products.

TREE STRUCTURES

When data is structured using linear constructs, items are arranged in a sequence, with each item following the next. These structures are well-suited for representing real-world scenarios that require lists, stacks, and queues. However, the real world often presents more complex situations where data items need to be connected and organized in ways beyond a simple linear sequence.

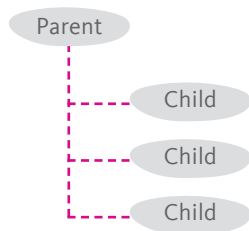
EXAMPLE 2.9

While the warehouse may rely on a stack or queue to efficiently manage inbound and outbound products, maintaining a full inventory of its stock requires a more complex data structure—one that can accommodate various categories and subcategories of products. The tree data structure is well-suited for this purpose, as it allows data to be organized hierarchically in levels that effectively support the categorization and subcategorization of items along branches.

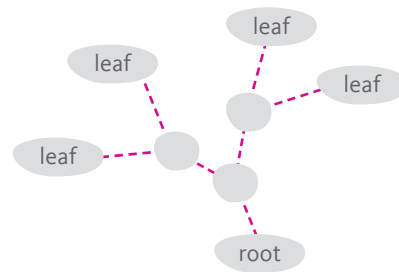


This is the organization chart for the food inventory at the warehouse where Jenna works. It is a tree structure with three hierarchical levels, each containing a category or subcategory of a food product.

In a tree structure, data items are referred to as nodes. A node functions as a data point that, in its most basic form, includes the data itself and additional information that points to other nodes to which this node is connected. Nodes that precede it are referred to as parent nodes, while nodes that follow are known as child nodes. The original parent node from which all other nodes originate is called the root node. Nodes at the very end of the tree structure, which do not have any child nodes, are referred to as leaf nodes.



A parent node with three child nodes.



A tree data structure with the root node and leaf nodes labeled.

A NODE IS A DATA CONSTRUCT THAT HOLDS BOTH DATA AND REFERENCES TO OTHER NODES.

Let's construct Jenna's warehouse food inventory as a tree data structure. The first step would be to create the file that would hold the dataset.

```

CREATE New Dataset
  Structure = Tree
  Size = Dynamic
  Data Type = Any
  Elements = []
  Node = (parent_index, data)
  Name = "food_inventory"
  Insert = any position
  Remove = any position
  
```

OUTPUT:
 food_inventory = []

These instructions create a dynamic file named food_inventory with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its parent nodes.

The instructions present a simplified method for defining a node. A tuple is used to contain the data, along with information about the parent nodes of the node being defined. It also employs the indexing method used in linear data structures to reference the position of each node within its hierarchical level and along its branch.

```
food_inventory = []
    INSERT node (R, "Food")
    PRINT food_inventory
END
```

OUTPUT:
Food

These instructions insert the root node, identified by the letter 'R', into an empty food_inventory tree and establish "Food" as the data associated with that node.

```
food_inventory = [(R, "Food")]
    INSERT node (R_0, "Meat")
    INSERT node (R_1, "Fruit")
    INSERT node (R_2, "Vegetable")
    PRINT food_inventory
END
```

OUTPUT:
Food
|____ Meat
|____ Fruit
|____ Vegetable

Three additional nodes are inserted as child nodes of the root, creating a new level in the hierarchy. Each node is indexed based on its position in this level, starting from 0.

```
food_inventory = [(R, "Food"),
    (R_0, "Meat"),
    (R_1, "Fruit"),
    (R_2, "Vegetable")
]
    INSERT node (R_1_0, "Apple")
    PRINT food_inventory
END
```

OUTPUT:
Food
|____ Meat
|____ Fruit
|____ |____ Apple
|____ Vegetable

These instructions further expand the food_inventory tree by adding a new child node under the existing "Fruit" node. The new node, labeled R_1_0 with the data "Apple," is inserted as a child of R_1 ("Fruit").

Removing data from a tree is straightforward when the data exists within a leaf node, meaning there are no other nodes dependent on it. However, if the data is within a parent node, removing it will likely result in all its dependent child nodes being deleted as well. This is because the child nodes rely on the parent node to maintain their position and relationship within the tree structure.

```
food_inventory = [(R,"Food"),
                  (R_0, "Meat"),
                  (R_1, "Fruit"),
                  (R_2, "Vegetable")
                  (R_1_0, "Apple"
                  ]
REMOVE node (R_1)
PRINT food_inventory
END
```

OUTPUT:

```
Food
|_____ Meat
|_____ Vegetable
```

The food_inventory tree initially contains nodes representing categories like “Meat,” “Fruit,” and “Vegetable.” When the instruction ‘REMOVE node (R_1)’ is executed, the “Fruit” node (R_1) is removed from the tree. As a result, all child nodes under “Fruit,” including “Apple” (R_1_o), are also deleted.

Nodes can change position in a tree as long as their child nodes are moved with them.

```
food_inventory = [(R,"Food"),
                  (R_0, "Meat"),
                  (R_1, "Fruit"),
                  (R_2, "Vegetable")
                  (R_1_0, "Apple"
                  ]
SWAP node (R_1),(R_2)
PRINT food_inventory
END
```

OUTPUT:

```
Food
|_____ Meat
|_____ Vegetable
|_____ Fruit
|           |_____ Apple
```

The instruction ‘SWAP node (R_1), (R_2)’ swaps the positions of the “Fruit” (R_1) and “Vegetable” (R_2) nodes. As a result, “Vegetable” will move into the position previously occupied by “Fruit,” and vice versa, while all their respective child nodes (such as “Apple” under “Fruit”) are moved along with them, maintaining the hierarchical integrity of the tree structure.

GRAPH STRUCTURES

In a tree, data is placed in nodes that are arranged along branches. Nodes can only connect to other nodes along their branch; they cannot directly connect to nodes in different branches. In contrast, in a graph, data is also placed in nodes, but the connections between nodes are not restricted. Nodes can interconnect freely, allowing for more complex relationships between data points.

In a tree data structure, we gave nodes a simplistic format that contains a reference to parent nodes and the data itself (`parent_index`, `data`). In a graph data structure, nodes have more complex connections, so a node will need to contain more information about linked nodes. It includes a list of all the indices of linked nodes, formatted as (`node_index`, `data`, `linked_node1`, `linked_node2`, ...).

EXAMPLE 2.10

Now that Jenna has used a queue to organize inbound and outbound food products, ensuring their freshness by minimizing shelf time, and has utilized a tree to create an inventory for all products in the warehouse, she now needs to manage the delivery process from the warehouse to her clients. For this task, she will need a graph to represent the various client destinations on the map.



Jenna's warehouse is in Seville, in the south of Spain. Her clients are in the cities of Madrid, Barcelona, and Valencia. Each of these four cities can be represented as a node in a graph data structure, and the roads connecting the cities can be represented as links between the nodes of the graph.

Just like with any data structure that uses nodes, nodes in a graph must be indexed. Typically, indexing is done using numbers, but that's not always the case—characters such as letters can also be used, as seen in the previous example where the letter R was used to refer to the root node. In a graph structure, there is no root node, and in fact, any node can act as a root. In this example, we will use letters to index the nodes. The letters A, B, C, and D will refer to the nodes representing the cities of Seville, Madrid, Barcelona, and Valencia, respectively.

One method to approach constructing a graph is by creating a list of each node and every node connected to it, known as an adjacency list. In this example, each node represents a city, and the cities are connected by roads. For the sake of this example, we will assume that the road between Madrid and Valencia is still under construction and cannot be used or included.

Adjacency List using City Names:

Seville is connected to: Madrid, Barcelona, and Valencia
Madrid is connected to: Seville and Barcelona
Barcelona is connected to: Seville, Madrid, and Valencia
Valencia is connected to: Seville and Barcelona

Adjacency List using Indices:

A: [B, C, D]
B: [A, C]
C: [A, B, D]
D: [A, C]

But graphs can carry more information—not just the data at each node, such as the name of the city, but also details about the links between nodes, referred to as ‘edges’. In this example, the edges between nodes could represent data such as the distance between cities. This additional information allows the graph to model real-world scenarios more accurately, where the relationships between entities (like cities) are as important as the entities themselves.

IN A GRAPH, AN **EDGE** IS A LINK BETWEEN TWO NODES THAT CARRIES ADDITIONAL INFORMATION ABOUT THEIR RELATIONSHIP.

Distances between cities:

Seville - Madrid:	538 km
Seville - Barcelona:	1,031 km
Seville - Valencia:	653 km
Madrid - Barcelona:	621 km
Valencia - Barcelona:	351 km

Numerical Value Assigned to Each Edge:

AB, BA = 538
AC, CA = 1,031
AD, DA = 653
BC, CB = 621
CD, DC = 351

This new data about the distances between cities will be included as additional information regarding the edge between each node. Accordingly, the adjacency list will be updated to include the edge data.

Updated Adjacency List:

A: [(B, 538), (C, 1,031), (D, 653)]
 B: [(A, 538), (C, 1,031)]
 C: [(A, 1,031), (B, 621), (D, 351)]
 D: [(A, 653), (C, 351)]

To construct Jenna's delivery map as a graph data structure, the first step is to create the file that will hold the dataset.

```
CREATE New Dataset
  Structure = Graph
  Size = Dynamic
  Data Type = Any
  Elements = []
  Node = (node_index, data, (linked_node, edge))
  Name = "delivery_map"
  Insert = any position
  Remove = any position
```

OUTPUT:
 delivery_map = []

These instructions create a dynamic file named delivery_map with no initial elements and allow data of any type to be inserted or removed from any position. The instructions also define the format of each node to contain the data and identify its connected nodes and their edges.

Once the graph has been created, the first node can be inserted. Since there are no other nodes to link to, the first node would be defined as (A, “Seville”), with A as the index of the node and “Seville” as the data it contains.

```
delivery_map = []
    INSERT node (A,“Seville”)
END
```

OUTPUT:
`delivery_map = [(A,“Seville”)]`

These instructions insert the first node, identified by the letter ‘A’, into an empty `delivery_map` graph and associate the city name “Seville” with that node.

Since there are no other nodes yet, no additional data, such as node indices or edge information, can be included at this stage.

The next step is to insert the remaining three nodes that represent the cities of Madrid, Valencia, and Barcelona, along with the distances between them as values for the edges connecting the nodes.

```
delivery_map = [(A,“Seville”)]
    INSERT node (B,“Madrid”,(A,538))
    INSERT node (C,“Barcelona”,(A,1031))
    INSERT node (D,“Valencia”,(A,653))
END
```

OUTPUT:
`delivery_map = [
 (A,“Seville”,(B,538),(C,1031),(D,653)),
 (B,“Madrid”,(A,538))
 (C,“Barcelona”,(A,1031))
 (D,“Valencia”,(A,653))
]`

With the insertion of the three nodes “Madrid” (B), “Barcelona” (C), and “Valencia” (D), the graph now includes all four nodes, with each node identifying the nodes it is linked to and their corresponding edges.

Note that the A node is automatically updated to include these connections, reflecting the distances to each city.

If Jenna wants to consider the distance her delivery truck needs to travel to deliver to her clients in Madrid, Barcelona, and Valencia, she can have the computer system perform the necessary calculations now that the dataset has been defined.

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538))
    (C, "Barcelona", (A, 1031))
    (D, "Valencia", (A, 653))
]
Trip_1 = AB + BA
Trip_2 = AC + CA
Trip_3 = AD + DA
Trip_All = Trip_1 + Trip_2 + Trip_3
PRINT Trip_All
END

```

OUTPUT:
4444

These instructions calculate the total distance Jenna's delivery truck would travel to visit each client in Madrid, Barcelona, and Valencia, and then return to Seville.

The distances for each round trip (Seville to Madrid and back, Seville to Barcelona and back, and Seville to Valencia and back) are summed, resulting in a total trip distance of 4,444 km, which is then printed as the final output.

Jenna realized that the total distance of 4,444 km is quite long. She decided to consider an alternative delivery route that started in Seville, passed through each of the three delivery destinations—Madrid, Barcelona, and Valencia—before returning to the warehouse in Seville. To calculate this route, she would need to include the distances between Madrid and Barcelona, as well as between Barcelona and Valencia.

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538))
    (C, "Barcelona", (A, 1031))
    (D, "Valencia", (A, 653))
]
INSERT edge (B:C, 621)
INSERT edge (C:D, 351)
PRINT delivery_map
END

```

OUTPUT:

```

delivery_map = [
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),
    (B, "Madrid", (A, 538), (C, 621)),
    (C, "Barcelona", (A, 1031), (B, 621), (D, 351)),
    (D, "Valencia", (A, 653), (C, 351))
]

```

The instructions update the delivery_map by inserting edges between Madrid (B) and Barcelona (C) with a distance of 621 km, and between Barcelona (C) and Valencia (D) with a distance of 351 km.

Now that the required data has been entered into the graph, Jenna can have the distance for the alternate route calculated.

```
delivery_map = [  
    (A, "Seville", (B, 538), (C, 1031), (D, 653)),  
    (B, "Madrid", (A, 538), (C, 621)),  
    (C, "Barcelona", (A, 1031), (B, 621), (D, 351)),  
    (D, "Valencia", (A, 653), (C, 351))  
]  
Trip_All = AB + BC + CD + DA  
PRINT Trip_All  
END
```

OUTPUT:
2163

The instructions calculate the distance of the alternate route by summing the values of the edges between each of the four nodes: AB, BC, CD, and DA. The calculated distance for this route is 2,163 km, significantly less than the initial route, making it a more efficient option for deliveries.

The use of a graph helped organize the data in a way that allowed for the calculations Jenna needed to make an informed decision on which route to take. At the warehouse, Jenna used a linear queue to manage inbound and outbound products, a tree to manage inventory, and a graph to manage delivery routes. These simple examples illustrate how data structures are crucial in the process of converting data into actionable information.

CONCLUSION

In conclusion, this lesson and the examples provided emphasize that the selection of a data structure is not arbitrary; it must be carefully aligned with the nature of the data and the specific requirements of the task at hand. Linear structures, such as arrays, lists, and stacks, are straightforward and efficient for managing sequential data, while trees and graphs offer the complexity needed to handle more complex relationships, such as hierarchical or interconnected data. Therefore, it is crucial to select the most appropriate structure to accurately represent and manage the underlying data.

SUMMARY

Data structures are specialized formats for organizing, retrieving, modifying, processing, and storing data.

Algorithms control these data structures by defining the rules for size, data type, and methods for inserting, removing, and accessing items.

Data structures are broadly classified into linear (array, list, tuple, stack, and queue) and non-linear (tree and graph) categories.

In linear data structures, data is arranged sequentially.

- Array: Fixed size, accepts one data type, and does not allow the insertion or removal of data after creation.
- List: Dynamic size, accepts mixed data types, and allows insertion or removal of data from any position.
- Tuple: Fixed size and format, accepts mixed data types, but does not allow the insertion or removal of data after creation.
- Stack: Dynamic size, accepts mixed data types, and allows data to be added or removed only from the end, following the Last In, First Out (LIFO) principle.
- Queue: Dynamic size, accepts mixed data types, allows data to be added at the end and removed from the beginning, following the First In, First Out (FIFO) principle.

In non-linear data structures like trees and graphs, data is held in nodes that link data together.

- Tree: Nodes are linked to form a hierarchical structure of parent-child relationships arranged in branches.
- Graph: Nodes can connect freely, allowing more complex relationships between data points.

A node is a data construct that contains both data and references to other nodes, making it essential in organizing and navigating data in non-linear structures.

In a graph, an edge is a link between two nodes that carries additional information about their relationship.

Activity 2.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>F</u>	An array can hold multiple data types within the same structure.
<u>T</u>	A list has a dynamic size, meaning you can add or remove items from it.
<u>F</u>	A stack follows the First In, First Out (FIFO) principle.
<u>T</u>	In a tree structure, the root node is the node from which all other nodes descend.
<u>F</u>	A queue allows data to be added at any position and removed from any position.
<u>T</u>	In a graph, an edge represents a connection between two nodes and can carry additional information.
<u>F</u>	Tuples can have a dynamic size and allow for the addition of new data after creation.
<u>F</u>	Nodes in a tree can connect directly to nodes in different branches.
<u>T</u>	The Last In, First Out (LIFO) principle is applied when using a stack data structure.
<u>F</u>	A graph structure is more suitable than a tree structure for representing hierarchical data.

Activity 2.2

Answer the questions below.

1. Which data structure follows the Last In, First Out (LIFO) principle?
 - a) Queue
 - b) **Stack**
 - c) List
 - d) Array
2. Which is a characteristic of an array?
 - a) It can hold multiple data types
 - b) It has a dynamic size
 - c) **It can only hold one data type**
 - d) It allows insertion and deletion of data at any position
3. In a tree structure, what is the topmost node called?
 - a) Leaf node
 - b) Child node
 - c) **Root node**
 - d) Parent node
4. Which data structure is most suitable for managing hierarchical data?
 - a) **Tree**
 - b) Graph
 - c) Array
 - d) Stack
5. What is the primary feature of a tuple?
 - a) It has a dynamic size
 - b) It follows the First In, First Out (FIFO) principle
 - c) It can be modified after creation
 - d) **It has a fixed format**

Activity 2.3

Answer the questions below.

1. Which data structure follows the First In, First Out (FIFO) principle?
 - a) Stack
 - b) Queue
 - c) Tuple
 - d) List
2. What does an edge represent in a graph?
 - a) The data stored in a node
 - b) The hierarchical level of a node
 - c) The connection between two nodes
 - d) The index of a node
3. Which of these data structures can freely connect nodes?
 - a) Tree
 - b) Graph
 - c) Stack
 - d) Queue
4. Which data structure is best suited for representing a subway network?
 - a) Tree
 - b) Array
 - c) Stack
 - d) Graph
5. What happens when a node with child nodes is removed from a tree structure?
 - a) The tree collapses.
 - b) The child nodes become root nodes.
 - c) The tree becomes a graph.
 - d) The child nodes are automatically removed.

Activity 2.4

Match each data structure with the statement that best describes it.

Queue	Graph	Stack
Tree	Tuple	List

1. A fixed-size, ordered collection of the same data type. List
2. A hierarchical structure where each element is a node. Tree
3. A structure that follows the FIFO principle. Queue
4. It has a fixed size and fixed format. Tuple
5. A structure that follows the LIFO principle. Stack
6. A flexible structure where nodes are connected by edges. Graph

Activity 2.5

Match each data structure with the task for which it is best optimized.

Tree	Array	Tuple
Queue	Stack	Graph

1. Handling tasks in the order they arrive, like print jobs. Queue
2. Modeling networks, like social connections. Graph
3. Organizing a file system, like a computer's directory. Tree
4. Storing the coordinates of a point in a 2D space as (x, y). Tuple
5. Storing a city's monthly average temperatures for a year. Array
6. Managing undo operations in a software application. Stack

Activity 2.6

Answer the questions below about the following dataset, which represents the average monthly temperatures (in °C) for a city over a year.

```
monthly_temp = [2.5, 3.0, 7.8, 12.5, 16.4, 20.1, 23.3, 22.8, 18.5, 13.2, 8.3, 4.1]
```

1. How many data items are in the array? 12
2. What type of data is in the array? numeric
3. What data item is in the fourth position? 12.5
4. What was the temperature in September? 18.5
5. What would be the output of this instruction? 8.3

```
PRINT index (10)
```

6. Assume there was an error with the average temperature for February—it is actually 3.2°C rather than 3.0°C. Review the following instructions and determine which ones can be implemented and which cannot. Explain.

```
REMOVE index (1) = 3.0  
INSERT index (1) = 3.2
```

```
UPDATE index (1) = 3.2
```

The remove and insert instructions cannot be implemented because they
would change the size of the array, which is not allowed. However, updating
the value held at a particular position is allowed as it does not affect
the size of the array.

Activity 2.7

Answer the questions below about the following dataset, which represents a grocery shopping list.

```
grocery_list = ["Apples", "Bread", "Milk", "Eggs", "Cheese", "Tomatoes"]
```

- 1. How many data items are in the list? 6
- 2. What type of data is in the list? alphanumeric
- 3. What data item is in the third position? Milk
- 4. What is the index of the data item "Cheese"? 4

5. Fill in the blanks with the data items from the list and write the index beneath each.

<u>Apples</u>	<u>Bread</u>	<u>Milk</u>	<u>Eggs</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>

6. Assume the eggs were removed from the grocery cart. To modify the list, you must:
- a) Remove all data items before and including index (3).
 - b) Remove all data items after and including index (3).
 - c) Remove the data item at index (3).
- Update the list accordingly.

<u>Apples</u>	<u>Bread</u>	<u>Milk</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>

7. Pears were added to the grocery cart. Write a pseudocode instruction that will add "Pears" to the list to follow "Apples". Then update the list.

INSERT index (1) = "Pears"

<u>Apples</u>	<u>Pears</u>	<u>Bread</u>	<u>Milk</u>	<u>Cheese</u>	<u>Tomatoes</u>
<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	

Activity 2.8

Answer the questions below about the undo stack of a text editor.

```
undo_stack = [
    "Italicize 'World'",
    "Type 'World'",
    "Bold 'Hello'",
    "Type 'Hello'"
]
```

1. How many actions were taken by the user? 4
2. What was the first action taken by the user? Type 'Hello'
3. What was the last action taken by the user? Italicize 'World'
4. What action is held at index (1)? Type 'World'
5. To undo "Bold 'Hello'", which actions must be undone? Write them in the order in which they are removed from the stack.

The last action taken will be removed first: "Italicize 'World'".

The next action to be removed is: "Type 'World'".

Finally, after removing the previous two actions, you can undo "Bold 'Hello'".

6. How many are left in the stack? 1
7. What is the item left in the stack? Type 'Hello'
8. The user now performs the following actions in this order: italicizes 'Hello', types 'World', italicizes 'World', underlines 'Hello World'. Rewrite the items in the order in which they would appear in the stack. Indicate the index of each.

Index (0) - Underlines 'Hello World'

Index (1) - Italicizes 'World'

Index (2) - Types 'World'

Index (3) - Italicizes 'Hello'

Index (4) - Type 'Hello'

Activity 2.9

Answer the questions below about the queue of customers waiting to speak with a customer service representative.

```
customer_queue = ["Maya", "Lee", "Chen", "Sara", "Omar"]
```

1. How many customers are in the queue? 5
2. Who was the first customer to enter the queue? Maya
3. Who was the last customer to enter the queue? Omar
4. Which customer will be the first to be served? Maya
5. From which index will customer names leave the queue? 0
6. Write the updated queue after the next customer is served.
customer_queue = ["Lee", "Chen", "Sara", "Omar"]
7. Which customer will be served next? Lee
8. Write the updated queue after the Lee is served.
customer_queue = ["Chen", "Sara", "Omar"]
9. Jenna joins the queue. Write the updated queue.
customer_queue = ["Chen", "Sara", "Omar", "Jenna"]
10. At which index is "Jenna"? 3
11. Read the following instruction and determine how it would be handled.

```
INSERT index (1) = "Mike"
```

A queue operates based on the FIFO principle, so any new data must be added at the end. This instruction will result in an error and cannot be carried out.

Activity 2.10

Answer the questions about the following list of tuples.

```
student_grades = [
    ("Maya", 85),
    ("Lee", 92),
    ("Sara", 78),
]
```

- How many tuples are in the list? 3
- What format does each tuple have? (name, grade)
- Which tuple is at index (0)? ("Maya", 85)
- What is Sara's grade? 78
- Read the following instruction and update the list accordingly.

```
INSERT index (1) = ("Mike", 83)
```

student_grades = [("Maya", 85), ("Mike", 83), ("Lee", 92), ("Sara", 78)]

- Assume there was an error entering Lee's grade. To modify the data, you must:
 - Swap the incorrect grade with the correct one inside the tuple at index (2).
 - Remove the tuple at index (2) and insert a corrected tuple at index (2).
 - Remove all tuples before index (2) before inserting the correct tuple.

Lee's correct grade is 91. Write the pseudocode instructions that will carry out this correction.

REMOVE index (2) = ("Lee", 92)

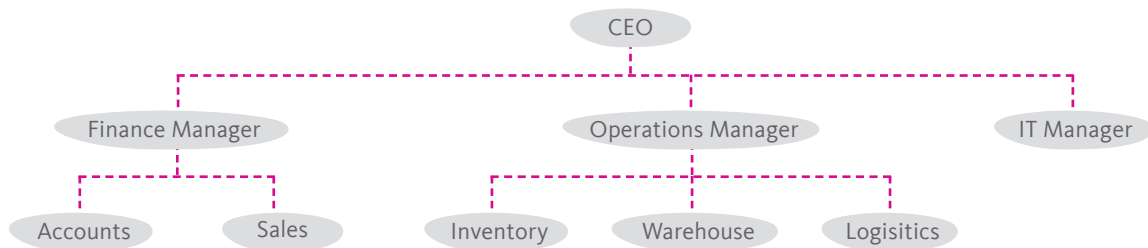
INSERT index (2) = ("Lee", 91)

Write the corrected list.

student_grades = [("Maya", 85), ("Mike", 83), ("Lee", 91), ("Sara", 78)]

Activity 2.11

Answer the questions below about the organization chart of the company where Jenna works.



1. Which data structure is best suited for this chart? Tree
2. How many hierarchical levels are in this chart? 3
3. Which data item is at the root node? CEO
4. How many child nodes does the root node have? 3
5. How many child nodes does the 'Finance Manager' have? 2
6. How many leaf nodes are there in total? 6
7. What are the leaf nodes under 'Finance Manager'? Accounts, Sales
8. Is 'IT Manager' a leaf node or a parent node? Leaf node
9. Using pseudocode, write the tree data structure for the first two levels of the hierarchy. Name it 'org_chart'.

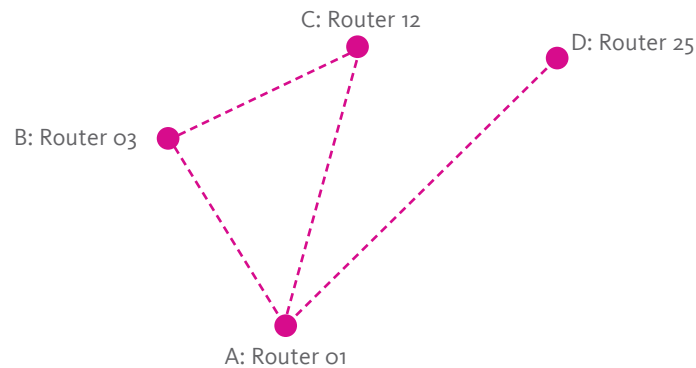
org_chart = [(R, "CEO"), (R_o, "Finance Manager"), (R_1, "Operations Manager"),
(R_2, "IT Manager")]

10. Write a pseudocode instruction that will add 'Marketing' to follow 'Sales' under 'Finance Manager'.

INSERT node (R_o_2, "Marketing")

Activity 2.12

Answer the questions about the following network of routers.



1. What type of data structure best represents this network? Graph
2. How many nodes are in this graph? 4
3. What is the name of the router designated as node B? Router 03
4. What is the node index of "Router 12"? C
5. Using node indices, write an adjacency list for this graph.

A: [B, C, D]
B: [A, C]
C: [A, B]
D: [A]
6. If the time it takes for a signal to travel between nodes (latency) is: A to B 12ms, A to C 18ms, A to D 25ms, and B to C 9ms, rewrite the adjacency list to include latency data as edges.

A: [(B, 12ms), (C, 18ms), (D, 25ms)]
B: [(A, 12ms), (C, 9ms)]
C: [(A, 18ms), (B, 9ms)]
D: [(A, 25ms)]

